BlackRidge Platforms — Complete Security Fix Prompt for Replit Agent

Based on actual QA audit findings — Score: 38/100 → Target: 90+/100

Stack: Node.js + Express.js backend + React/Vite frontend (single server)

PASTE THIS ENTIRE PROMPT INTO REPLIT AGENT

My Express.js + React/Vite site at blackridgeplatforms.com received a security audit score of 38/100. I need you to fix every issue found. The site runs on a single Node.js server where Express handles all API routes and also serves the Vite-built React frontend as static files.

Here is every finding from the audit report with the exact fix required for each one:

HIGH SEVERITY FIXES (Fix these first — they are the most critical)

HIGH-1: Missing Security Headers

Finding: The following headers are missing from all responses: `x-frame-options`, `x-content-type-options`, `content-security-policy`, `x-xss-protection`. Only `strict-transport-security` is present.

Fix: Install helmet and add it as the first middleware in the main server file (likely `server.js`, `index.js`, or `app.js`):

Bash

```
npm install helmet
```

JavaScript

```
const helmet = require('helmet');

// Add this as the VERY FIRST middleware, before anything else
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "'unsafe-inline'", "'unsafe-eval'"],
      styleSrc: ["'self'", "'unsafe-inline'", "https://fonts.googleapis.com"],
      fontSrc: ["'self'", "https://fonts.gstatic.com", "data:"],
      imgSrc: ["'self'", "data:", "https:", "blob:"],
      connectSrc: ["'self'", "https:"],
      frameSrc: ["'none'"],
      objectSrc: ["'none'"],
      upgradeInsecureRequests: [],
    },
  },
  crossOriginEmbedderPolicy: false,
  xFrameOptions: { action: 'sameorigin' },
  xContentTypeOptions: true,
  xXssProtection: true,
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
} ));
```

HIGH-2: Authentication Bypass on 4 Endpoints

Finding: These endpoints returned HTTP 200 without any authentication token:

- `GET /api/users` → returned 200
- `GET /api/admin` → returned 200
- `GET /api/dashboard` → returned 200
- `GET /api/profile` → returned 200

Fix: Create a file called `middleware/auth.js` (create the `middleware` folder if it does not exist):

JavaScript

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

const JWT_SECRET = process.env.JWT_SECRET || process.env.SESSION_SECRET;

function requireAuth(req, res, next) {
  // Support Bearer token in Authorization header
  const authHeader = req.headers['authorization'];
```

```
  if (authHeader && authHeader.startsWith('Bearer ')) {
    const token = authHeader.slice(7);
    try {
      req.user = jwt.verify(token, JWT_SECRET);
      return next();
    } catch {
      return res.status(401).json({ error: 'Invalid or expired token' });
    }
  }

  // Support session cookie (connect.sid via express-session)
  if (req.session && req.session.userId) {
    req.user = { id: req.session.userId, role: req.session.userRole };
    return next();
  }

  return res.status(401).json({ error: 'Authentication required' });
}

function requireAdmin(req, res, next) {
  if (!req.user) return res.status(401).json({ error: 'Authentication required'
  if (req.user.role !== 'admin') return res.status(403).json({ error: 'Admin ac
  return next();
}

module.exports = { requireAuth, requireAdmin };
```

Then find where `/api/users`, `/api/admin`, `/api/dashboard`, and `/api/profile` are defined and add the middleware:

JavaScript

```
const { requireAuth, requireAdmin } = require('./middleware/auth');

// These routes MUST require authentication
app.get('/api/users', requireAuth, (req, res) => { /* existing code */ });
app.get('/api/profile', requireAuth, (req, res) => { /* existing code */ });
app.get('/api/dashboard', requireAuth, (req, res) => { /* existing code */ });

// Admin routes need both auth AND admin role check
app.get('/api/admin', requireAuth, requireAdmin, (req, res) => { /* existing co
app.use('/api/admin', requireAuth, requireAdmin); // covers all /api/admin/* su

// Also protect these additional routes found in the audit
app.get('/api/projects', requireAuth, (req, res) => { /* existing code */ });
app.get('/api/invoices', requireAuth, (req, res) => { /* existing code */ });
```

Do NOT add requireAuth to these — they must stay public:

- `POST /api/auth/login`
- `POST /api/auth/register`
- `GET /health`
- `GET /` and all static file serving

HIGH-3: No Rate Limiting on /api/auth/login

Finding: 15 rapid requests were sent to `/api/auth/login`. All 15 returned 200. No 429 response was ever triggered. This allows unlimited brute-force attacks.

Fix:

Bash

```bash
npm install express-rate-limit
```

JavaScript

```javascript
const rateLimit = require('express-rate-limit');

// Strict limiter for authentication endpoints
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 10, // max 10 attempts per IP per 15 minutes
  standardHeaders: true,
  legacyHeaders: false,
  handler: (req, res) => {
    res.status(429).json({
      error: 'Too many login attempts. Please try again in 15 minutes.',
      retryAfter: 900,
    });
  },
});

// General API limiter
const apiLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 100,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too many requests. Please slow down.' },
});
```

```
// Apply BEFORE route definitions
app.use('/api/auth/login', authLimiter);
app.use('/api/auth/register', authLimiter);
app.use('/api/auth/forgot-password', authLimiter);
app.use('/api/auth/reset-password', authLimiter);
app.use('/api/', apiLimiter);
```

MEDIUM SEVERITY FIXES

MEDIUM-1: Non-Existent Endpoint Returns 200 Instead of 404

Finding: A request to `/api/does-not-exist-qa-test` returned HTTP 200 in 20ms. This indicates a wildcard catch-all route is returning 200 for everything, which is incorrect behavior.

Fix: Find any wildcard route like `app.get('*', ...)` or `app.use('*', ...)` that serves the React frontend. It is likely at the bottom of your server file and looks like this:

JavaScript

```
// This is probably what you have now — it catches EVERYTHING including /api/*
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'dist', 'index.html'));
});
```

Replace it with this version that correctly returns 404 for unknown API routes:

JavaScript

```
// Return 404 for unknown /api/* routes — do NOT serve the React app for API pa
app.use('/api/*', (req, res) => {
  res.status(404).json({
    error: `Route ${req.method} ${req.path} not found`,
  });
});

// Only serve the React frontend for non-API routes (this must come AFTER the A
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'dist', 'index.html'));
});
```

MEDIUM-2: Invalid Email Accepted on /api/users

Finding: The server accepted `'not-an-email'` as a valid email address on `/api/users` and returned 200.

Fix:

```bash
npm install zod
```

Create `middleware/validate.js`:

```javascript
// middleware/validate.js
const { z } = require('zod');

function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(422).json({
        error: 'Validation failed',
        details: result.error.errors.map(e => ({
          field: e.path.join('.'),
          message: e.message,
        })),
      });
    }
    req.validatedBody = result.data;
    next();
  };
}

module.exports = { validate, z };
```

Apply email validation to `/api/users` and any other route that accepts an email:

```javascript
const { validate, z } = require('./middleware/validate');

const createUserSchema = z.object({
  name: z.string().min(1, 'Name is required').max(200),
  email: z.string().email('Invalid email address format'),
  password: z.string().min(8, 'Password must be at least 8 characters').max(128
  role: z.enum(['user', 'admin']).optional(),
});
```

```
const loginSchema = z.object({
  email: z.string().email('Invalid email address'),
  password: z.string().min(1, 'Password is required'),
});

// Apply to routes
app.post('/api/users', requireAuth, validate(createUserSchema), (req, res) => {
  const data = req.validatedBody; // use validated data
  // ... existing handler
});

app.post('/api/auth/login', authLimiter, validate(loginSchema), (req, res) => {
  const { email, password } = req.validatedBody;
  // ... existing handler
});
```

MEDIUM-3: Negative Numbers Accepted on /api/users (price/amount fields)

Finding: The server accepted negative values for price and amount fields on `/api/users` and returned 200. This can cause financial calculation errors.

Fix: Update your Zod schemas to reject negative numbers on any monetary or quantity field:

JavaScript

```
// Add these schemas for any route that handles money or quantities
const positiveNumber = z.number().positive('Value must be greater than zero');
const nonNegativeNumber = z.number().min(0, 'Value cannot be negative');

// Example for an invoice or order schema
const invoiceSchema = z.object({
  amount: z.number().positive('Amount must be greater than zero'),
  quantity: z.number().int().positive('Quantity must be a positive integer'),
  price: z.number().positive('Price must be greater than zero'),
  discount: z.number().min(0).max(100, 'Discount must be between 0 and 100').op
});

// Example for a project schema
const projectSchema = z.object({
  name: z.string().min(1).max(200),
  budget: z.number().min(0, 'Budget cannot be negative').optional(),
```

```
  hourlyRate: z.number().positive('Hourly rate must be greater than zero').opti
});
```

Find every route that accepts price, amount, budget, quantity, or rate fields and add `.positive()` or `.min(0)` validation to those fields.

MEDIUM-4: Unexpected 200 Response for Null/Missing Required Fields

Finding: When required fields were sent as null, the server returned 200 instead of 400 or 422.

Fix: This is solved by the same Zod validation middleware from MEDIUM-2. Make sure all required fields in your schemas do NOT have `.optional()` or `.nullable()` unless they are genuinely optional. Zod rejects null values for required fields by default:

JavaScript

```
// Required field — will reject null and undefined automatically
name: z.string().min(1, 'Name is required'),

// Optional field — explicitly mark with .optional()
description: z.string().optional(),

// Nullable field — only use when null is a valid business value
deletedAt: z.string().nullable().optional(),
```

Also add a global middleware to reject completely empty bodies on POST/PUT/PATCH routes:

JavaScript

```
app.use((req, res, next) => {
  if (['POST', 'PUT', 'PATCH'].includes(req.method)) {
    if (!req.body || Object.keys(req.body).length === 0) {
      return res.status(400).json({ error: 'Request body cannot be empty' });
    }
  }
  next();
});
```

LOW SEVERITY FIXES

LOW-1: No CORS Headers Detected

Finding: No `Access-Control-Allow-Origin` header was found on responses. This means cross-origin requests from your own frontend domain may fail in some browsers.

Fix:

Bash

```
npm install cors
```

JavaScript

```
const cors = require('cors');

const allowedOrigins = [
  'https://blackridgeplatforms.com',
  'https://www.blackridgeplatforms.com',
  // Add your Replit dev URL here too:
  process.env.REPL_SLUG ? `https://${process.env.REPL_SLUG}.${process.env.REPL_
  process.env.NODE_ENV !== 'production' ? 'http://localhost:3000' : null,
  process.env.NODE_ENV !== 'production' ? 'http://localhost:5173' : null,
].filter(Boolean );

app.use(cors({
  origin: (origin, callback) => {
    if (!origin) return callback(null, true); // allow non-browser requests
    if (allowedOrigins.includes(origin)) return callback(null, true);
    return callback(new Error('Not allowed by CORS'));
  },
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-Requested-With'],
}));
```

LOW-2: Large Payload (10MB) Returned 200

Finding: The server accepted a 10MB request body and returned 200. This leaves the server open to denial-of-service via oversized payloads.

Fix: Find where you set up `express.json()` and add a size limit:

JavaScript

```
// Replace your existing express.json() line with this:
app.use(express.json({ limit: '1mb' }));
```

```
app.use(express.urlencoded({ extended: true, limit: '1mb' }));

// Add an error handler specifically for payload size errors
app.use((err, req, res, next) => {
  if (err.type === 'entity.too.large') {
    return res.status(413).json({
      error: 'Request payload too large. Maximum size is 1MB.',
    });
  }
  next(err);
});
```

LOW-3: Missing JSON Content-Type on 3 Endpoints

Finding: `/api/users`, `/api/projects`, and `/api/invoices` are returning responses with `text/html; charset=utf-8` instead of `application/json`.

Fix: This is happening because these routes are returning HTML (likely the React index.html) instead of JSON. This is related to the wildcard route issue in MEDIUM-1. After fixing MEDIUM-1, these endpoints should return proper JSON. Additionally, add a global middleware to enforce JSON content type on all `/api/` responses:

JavaScript

```
// Force JSON content-type on all API routes
app.use('/api/', (req, res, next) => {
  res.setHeader('Content-Type', 'application/json');
  next();
});
```

LOW-4: No Method Enforcement on /api/users

Finding: `POST /api/users` returned 200 instead of 405 Method Not Allowed. If `/api/users` is a GET-only endpoint, it should reject POST requests.

Fix: Be explicit about which HTTP methods each route accepts. For routes that only support GET, add a handler for other methods:

JavaScript

```
// If /api/users only supports GET:
app.get('/api/users', requireAuth, (req, res) => {
  // ... existing GET handler
});
```

```
// Explicitly reject other methods on this route
app.all('/api/users', (req, res) => {
  res.status(405).json({
    error: `Method ${req.method} not allowed on this endpoint`,
    allowedMethods: ['GET'],
  });
});
```

If `/api/users` supports multiple methods (GET list, POST create), define each one explicitly:

JavaScript

```
app.get('/api/users', requireAuth, listUsersHandler);
app.post('/api/users', requireAuth, requireAdmin, validate(createUserSchema), c
// Any other method → 405
app.all('/api/users', (req, res) => {
  res.status(405).json({ error: `Method ${req.method} not allowed`, allowedMeth
});
```

LOW-5: No Pagination on List Endpoints

Finding: List endpoints (`/api/users` , `/api/projects` , `/api/invoices`) do not include pagination metadata (page, total, limit).

Fix: Create `utils/response.js` :

JavaScript

```
// utils/response.js
function paginatedResponse(res, items, total, page, limit) {
  return res.json({
    success: true,
    data: items,
    pagination: {
      page: parseInt(page),
      limit: parseInt(limit),
      total,
      totalPages: Math.ceil(total / limit),
      hasMore: parseInt(page) * parseInt(limit) < total,
    },
  });
}

function successResponse(res, data) {
  return res.json({ success: true, data });
```

```
}

module.exports = { paginatedResponse, successResponse };
```

Update list routes to support pagination:

JavaScript

```
const { paginatedResponse } = require('./utils/response');

app.get('/api/users', requireAuth, async (req, res) => {
  const page = Math.max(1, parseInt(req.query.page) || 1);
  const limit = Math.min(100, parseInt(req.query.limit) || 20);
  const offset = (page - 1) * limit;

  // Replace with your actual database query
  const items = await db.query('SELECT * FROM users LIMIT ? OFFSET ?', [limit,
  const [{ total }] = await db.query('SELECT COUNT(*) as total FROM users');

  return paginatedResponse(res, items, total, page, limit);
});
```

Apply the same pattern to `/api/projects` and `/api/invoices`.

LOW-6: No API Versioning

Finding: No version prefix in URL paths or version headers detected.

Fix: Add an `API-Version` header to all API responses and prefix new routes with `/api/v1/`. For backward compatibility, keep the existing `/api/` routes working but add the version header:

JavaScript

```
// Add version header to all API responses
app.use('/api/', (req, res, next) => {
  res.setHeader('API-Version', '1.0.0');
  res.setHeader('X-API-Version', '1.0.0');
  next();
});

// For new routes going forward, use /api/v1/ prefix
// Keep existing /api/ routes as aliases for backward compatibility
```

INFO ITEMS — These explain why Data Flow tests were skipped

The following INFO findings all show the same error: `Unexpected token '<', "<!DOCTYPE "... is not valid JSON`

This means the Data Flow agent tried to call `/api/users` (and other endpoints) to run create/read/update/delete tests, but received an HTML page instead of JSON. This is the same root cause as LOW-3 and MEDIUM-1 — the wildcard route is serving the React HTML page for API routes that don't exist or aren't authenticated.

These INFO items will automatically resolve after fixing MEDIUM-1 (wildcard route) and HIGH-2 (authentication). Once the API routes return proper JSON (either data or a 401 error), the data flow tests will be able to run properly.

The affected INFO items are:

- Empty body validation test skipped
- Error format test skipped
- Create-read test error
- Data type test error
- Update test error
- Delete test error
- Financial test error
- No timestamps found to validate
- Concurrent test error

WHAT ALREADY PASSED (Do not break these)

The audit confirmed these are working correctly — make sure your fixes do not break them:

- HTTP properly redirects to HTTPS (301) ✓
- No SQL injection vulnerabilities detected ✓
- SSL/TLS certificate is valid ✓
- Server responds within acceptable time ✓

COMPLETE INSTALL COMMAND

Run this once to install all required packages:

```bash
npm install helmet express-rate-limit cors zod
```

ORDER OF OPERATIONS

Apply the fixes in this exact order to avoid breaking anything:

1. Install all packages: `npm install helmet express-rate-limit cors zod`
2. Create `middleware/auth.js`
3. Create `middleware/validate.js`
4. Create `utils/response.js`
5. Open the main server file and add in this order at the top of the middleware stack:
   - `app.use(helmet(...))` — security headers
   - `app.use(cors(...))` — CORS
   - `app.use(express.json({ limit: '1mb' }))` — body size limit
   - Rate limiters on auth routes
   - Empty body rejection middleware
   - JSON content-type enforcement on `/api/`
6. Add `requireAuth` to the 4 unprotected endpoints
7. Fix the wildcard route to return 404 for unknown `/api/*` paths
8. Add Zod validation to `/api/users`, `/api/auth/login`, `/api/auth/register`
9. Add pagination to `/api/users`, `/api/projects`, `/api/invoices`
10. Add method enforcement (`app.all`) after each route definition
11. Add the global error handler as the very last `app.use()` call

GLOBAL ERROR HANDLER (add as the very last middleware)

```javascript
// Global error handler — MUST be the last app.use() in the file
app.use((err, req, res, next) => {
  console.error(`[${new Date().toISOString()}] Error on ${req.method} ${req.pat
```

```
  if (err.message && err.message.includes('CORS')) {
    return res.status(403).json({ error: 'CORS policy violation' });
  }
  if (err.type === 'entity.too.large') {
    return res.status(413).json({ error: 'Payload too large. Maximum 1MB.' });
  }
  if (err.name === 'JsonWebTokenError' || err.name === 'TokenExpiredError') {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
  if (err.name === 'ValidationError') {
    return res.status(422).json({ error: err.message });
  }

  const isDev = process.env.NODE_ENV !== 'production';
  return res.status(err.status || 500).json({
    error: isDev ? err.message : 'An internal server error occurred',
  });
});
```

EXPECTED SCORE AFTER ALL FIXES

Finding	Severity	Status After Fix
Missing 4 security headers	HIGH	Fixed by helmet ✓
Auth bypass on 4 endpoints	HIGH	Fixed by requireAuth middleware ✓
No rate limiting on /api/auth/login	HIGH	Fixed by express-rate-limit ✓
Non-existent endpoint returns 200	MEDIUM	Fixed by API 404 handler ✓
Invalid email accepted on /api/users	MEDIUM	Fixed by Zod email validation ✓
Negative numbers accepted	MEDIUM	Fixed by Zod .positive() validation ✓
200 response for null required fields	MEDIUM	Fixed by Zod required field validation ✓
No CORS headers	LOW	Fixed by cors() middleware ✓

Large payload accepted	LOW	Fixed by express.json({ limit: '1mb' }) ✓
Missing JSON Content-Type	LOW	Fixed by wildcard route fix + header middleware ✓
No method enforcement	LOW	Fixed by app.all() 405 handlers ✓
No pagination on list endpoints	LOW	Fixed by paginatedResponse helper ✓
No API versioning	LOW	Fixed by API-Version response header ✓
Data Flow tests (9 INFO items)	INFO	Will resolve once API routes return JSON ✓

Projected score after all fixes: 88–95/100 (Grade: A)

End of Prompt